

物体検知輪読#3

Vision transformer

Max vision transformer

工学部情報学科2回

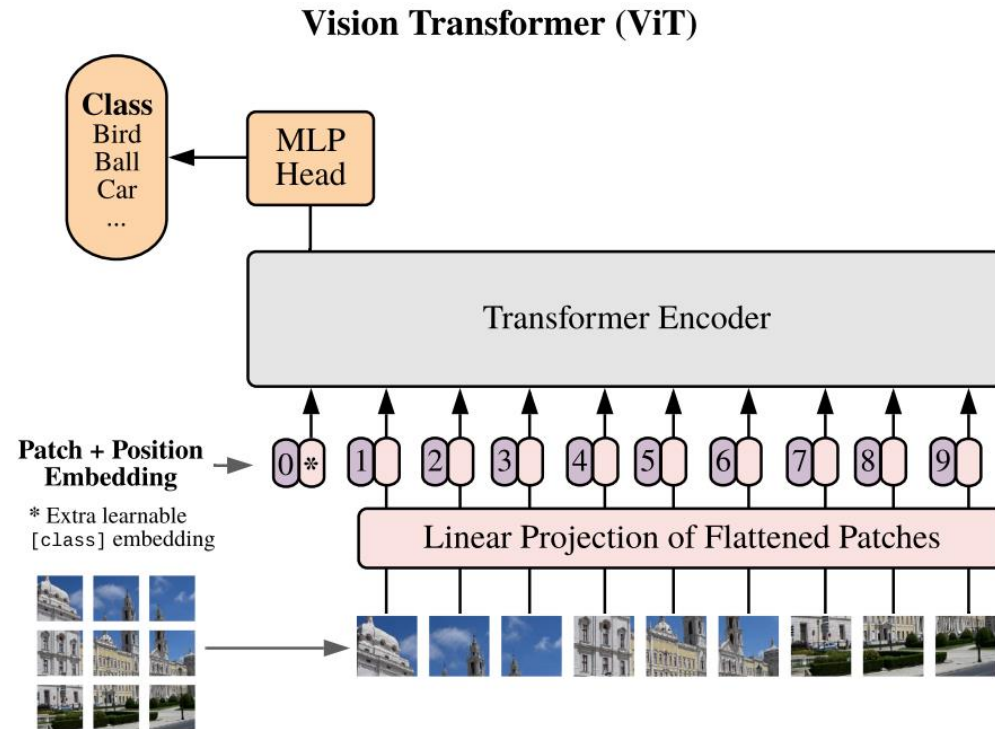
野村隆晃

- Vision transformer
(transformerの復習も兼ねて)
- Max vision transformer

- Vision transformer
(transformerの復習も兼ねて)
- Max vision transformer

1. Vision transformer

Vision transformerとは、transformerのencoderに画像をパッチ化して入力したものである。以下、transformerのアーキテクチャの復習がてら、vision transformerの構造を述べる。



画像の分類問題について

1. 画像をパッチとして分割
2. 各パッチを直列化してトークン化
3. Position embeddingを足す
4. Transformerのencodeに入力
5. Encoderの出力のうちclassトークンをMLPで分類

1.1 vision transformerの入力

画像をパッチごとのトークンに変換



$$\mathbb{R}^{H \times W \times C}$$

多くの場合で, $H = W$



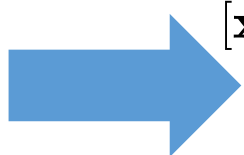
N_p 個のバッチに
分割



$$\mathbb{R}^{N_p \times P^2 \cdot C}$$

$$\mathbf{x}_p^1, \mathbf{x}_p^2, \dots, \mathbf{x}_p^{N_p}$$

各パッチが $p \times p$ の画像



埋め込み
ベクトル

$$[\mathbf{x}_{class}; \mathbf{x}_p^1 E; \mathbf{x}_p^2 E; \dots; \mathbf{x}_p^{N_p}] \in \mathbb{R}^{(N_p+1) \times D}$$

埋め込みの線形写像 E より
 D 次元のベクトルに変換して、
class トークンを追加

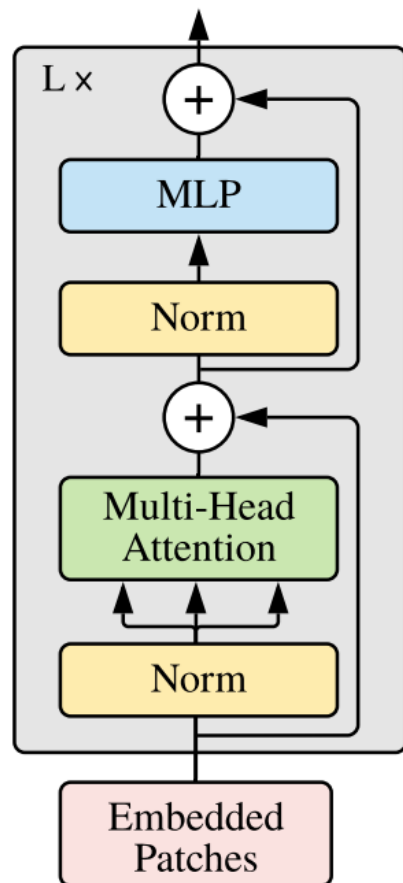


$$[\mathbf{x}_{class}; \mathbf{x}_p^1 E; \mathbf{x}_p^2 E; \dots; \mathbf{x}_p^{N_p}] + E_{pos} \in \mathbb{R}^{(N_p+1) \times D}$$

1.2 Transformerのencoder

TransformerのEncoderに D 次元の埋め込みベクトルを $(N_p + 1)$ 本入力する。

Transformer Encoder



$$\mathbf{z}'_{\ell} = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \quad \ell = 1 \dots L$$

$$\mathbf{z}_{\ell} = \text{MLP}(\text{LN}(\mathbf{z}'_{\ell})) + \mathbf{z}'_{\ell}, \quad \ell = 1 \dots L$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0)$$

MSA: Multi-head self attention

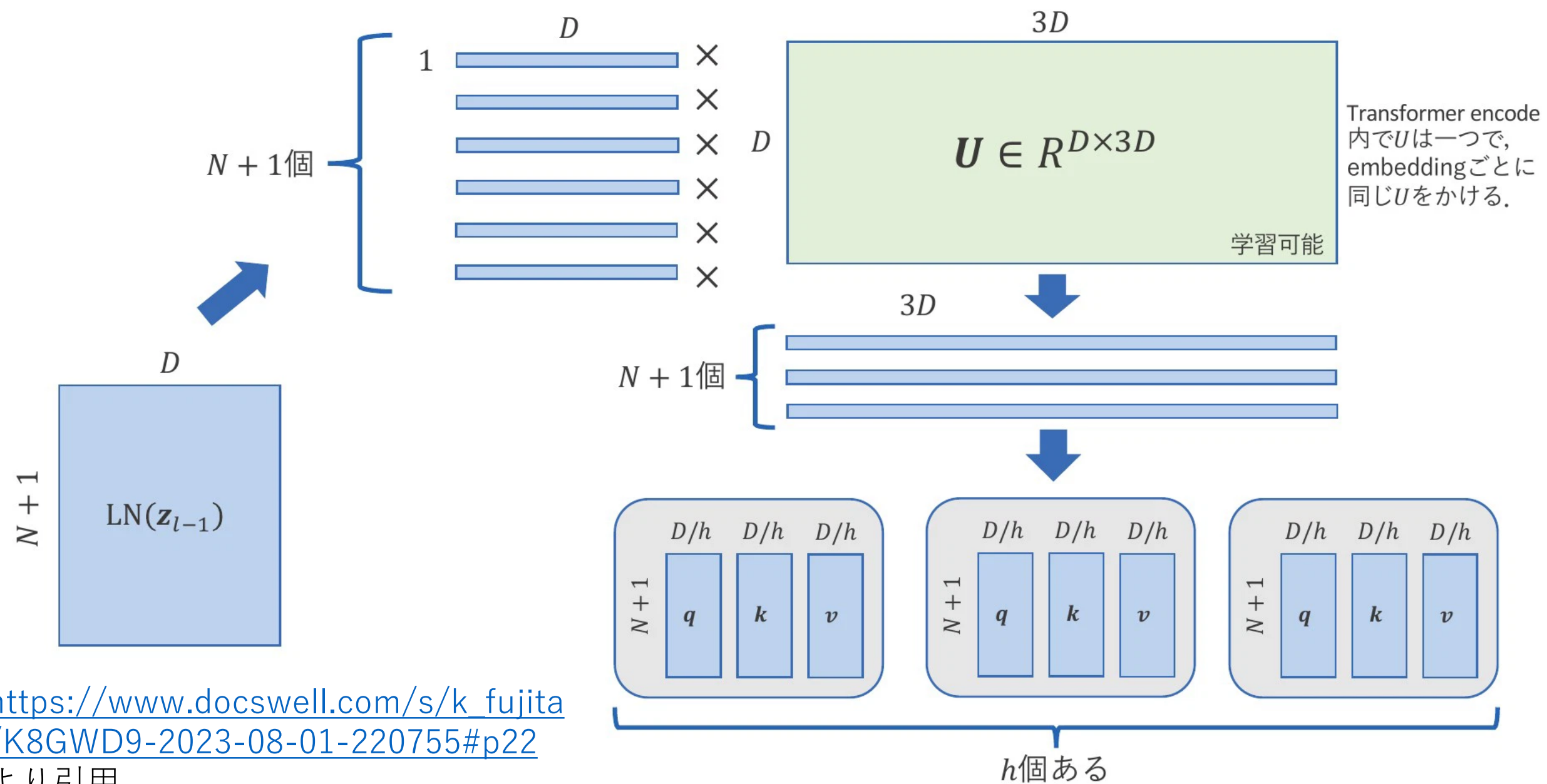
MLP: Multi layer perceptron

LN : Layer normalization

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Table 1: Details of Vision Transformer model variants.

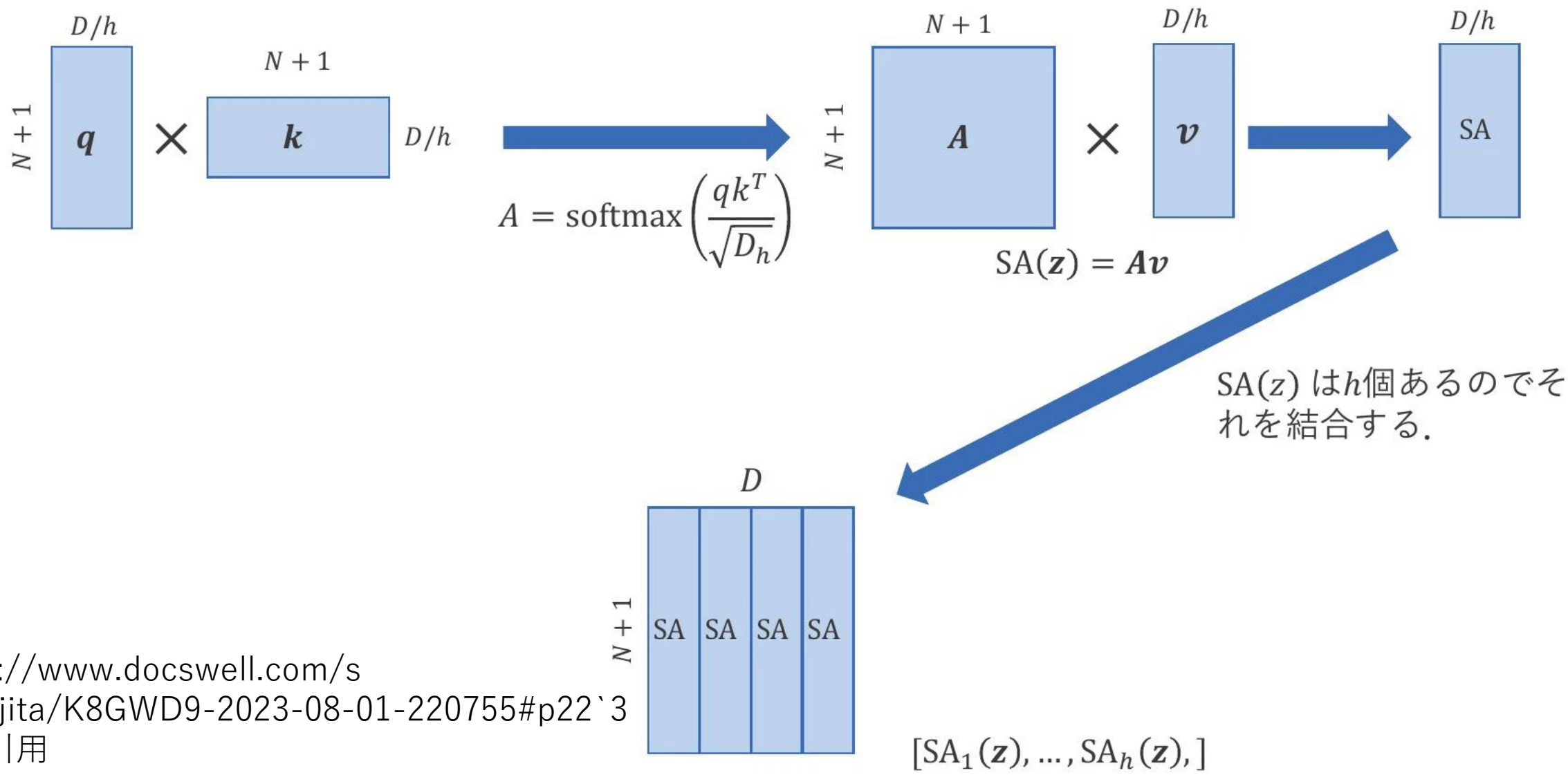
Multi-head self attention



https://www.docswell.com/s/k_fujita/K8GWD9-2023-08-01-220755#p22

より引用

Multi-head self attention

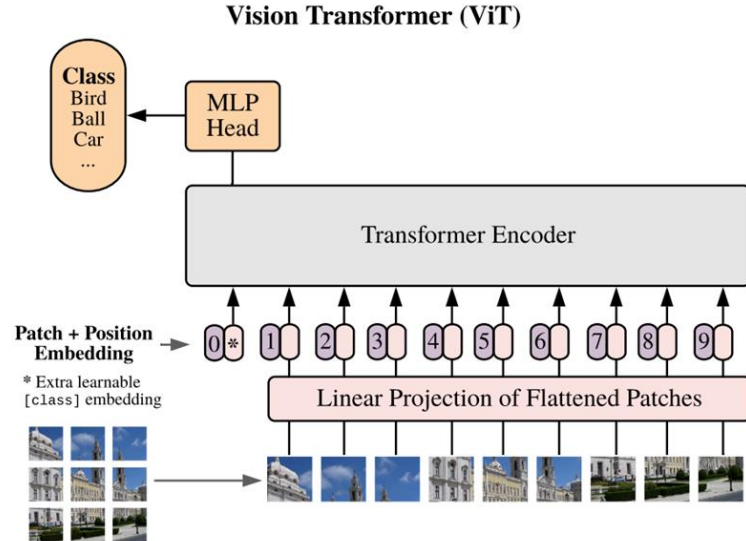


1.2 Transformerのencoder

[CLASS] トークンについて

学習可能なパラメータで、0番目のパッチ化された画像の埋め込みベクトル $\mathbf{x}_0$ として、挿入される。

画像分類問題では、CLASSトークンの埋め込みベクトルがMLPに入力され、予測クラスが決定される。最初は乱数で初期化されている。



$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0)$$

1.3 vision transformerの初期化

Vision transformerの訓練可能なパラメータは以下である。

1. CLASSトークン
2. 位置エンコーディング(1次元)
3. 埋め込みで用いる線形写像E
4. MSAでのQ,K,Vへの埋め込みの線形写像

2については、EをCNNの特微量マップへ置き換えるHybridモデルが提案されており、その場合は、EをResNetの特微量マップに置き換えて、そのパラメータで初期化している。

3はBERTモデルのパラメータをそのまま用いている。(base, large)

1.4 ViTの能力

ViTは画像分類タスクでSOTAを達成した。

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet Real	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

JFTはGoogleが保有する3億枚の画像の非公開データセット

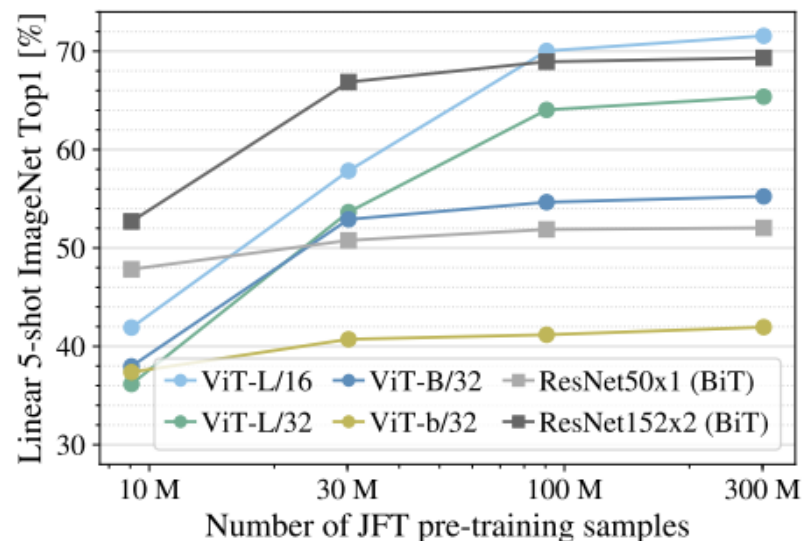
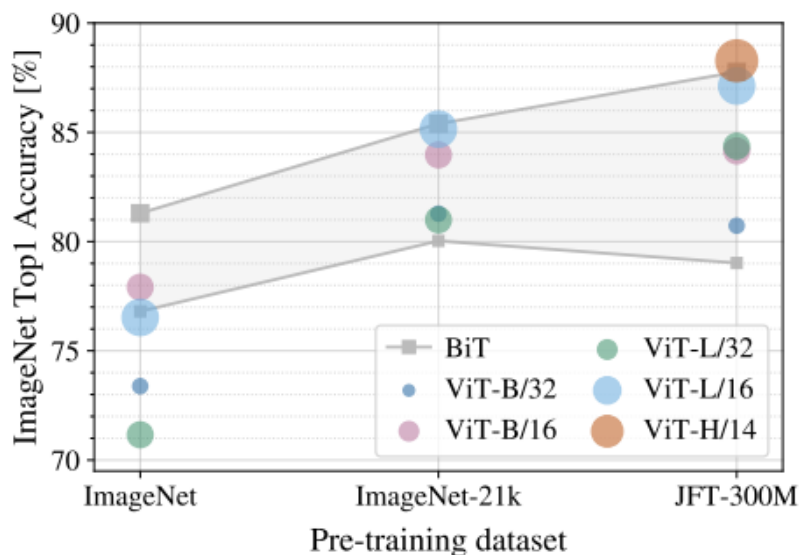
1.5 ViTのinductive biasの影響

ViTはinductive biasの欠如により、大規模なデータセットでの訓練が必要であることが、示されている。

Inductive biasとは？

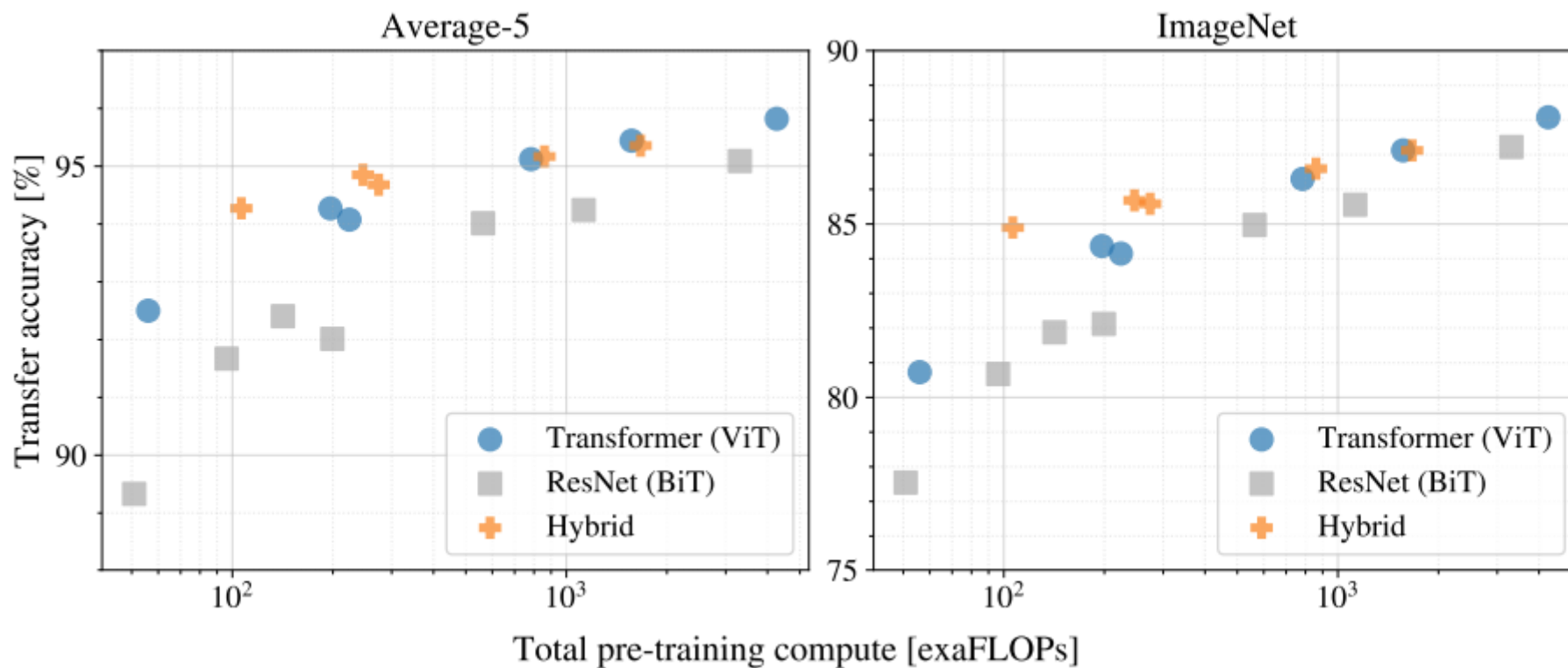
CNNは局所的に、2次元の近傍構造および並進移動に対しての等価性があるため、物体の特徴を捉えることができる。ただし、ViTは空間的な関係を一から学習しなければならない。

そのため、小規模なデータセットではCNNより劣る



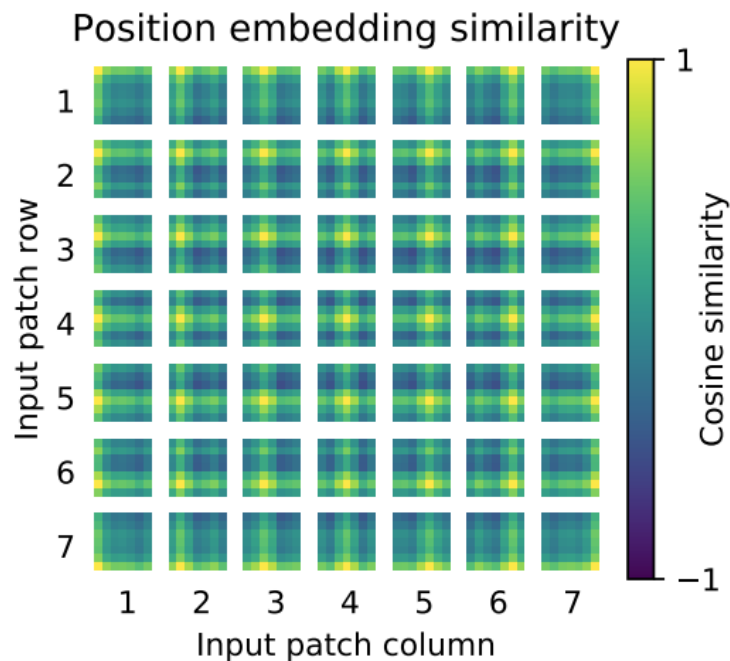
1.5 ViTのinductive biasの影響

計算機資源についても、データセットと同様に、低計算機資源でのViTのパフォーマンスの低さがみられる。しかし、Convとのhybridモデルは低予算でも十分な性能を発揮している

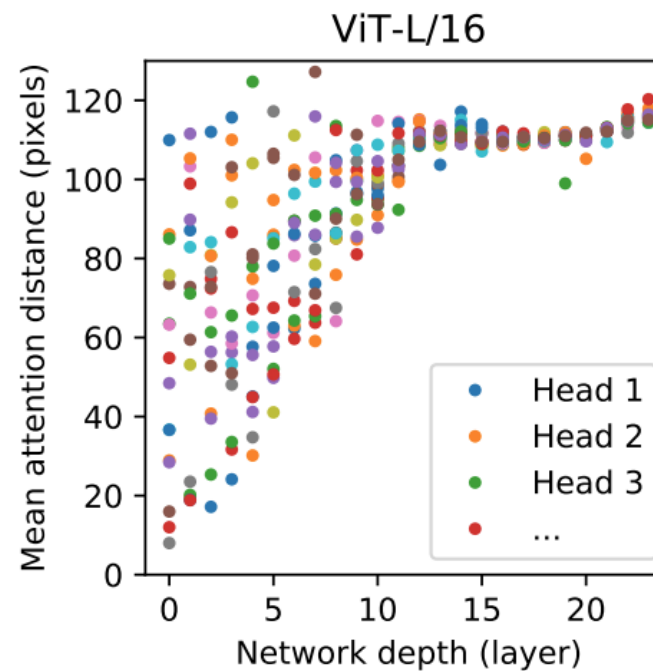


1.6 もろもろの学習結果

学習した結果の図



パッチ間の距離を正しく認識していることが分かる。



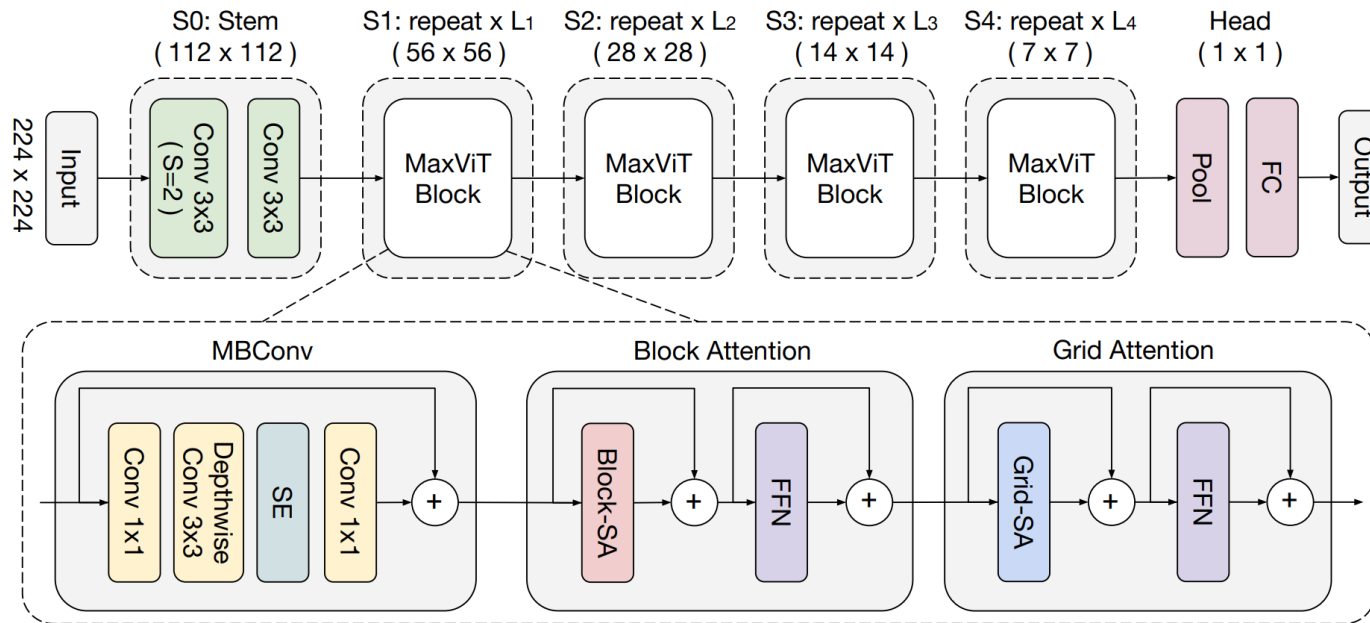
浅い層でも、高いattention distanceをもつヘッドが存在している。

- Vision transformer
(transformerの復習も兼ねて)
- Max vision transformer

2.1 Max Vision Transformer

ViTからの主な改善点は以下の3つ

1. Position embeddingがrelative attentionに代替
2. Multi Axis vision transformer
3. 画像がfeature mapとして入力
4. MBConvの導入

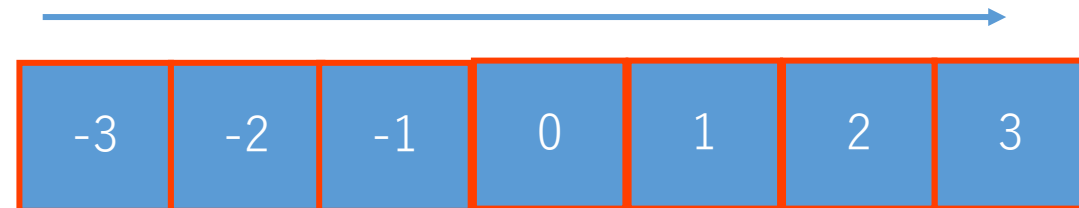


2.2 Relative attention

Relative attentionについて、MaxViTはposition embeddingが加算されていない

$$\text{RelAttention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d} + B)V,$$

縦横それぞれに対して、ある要素の前後を決定する。長さNだと、-N+1からN-1

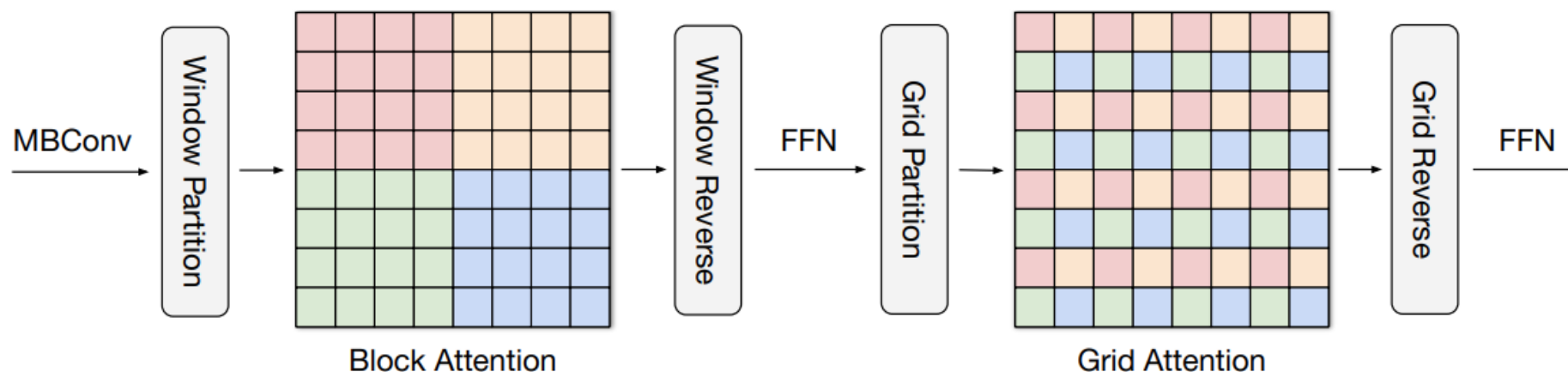


高さHの画像について、相対位置は(-H+1)から(H-1)であり、幅も同様にして、各要素に学習可能なパラメータが格納された行列 $\hat{B} \in R^{2H-1 \times 2W-1}$ を参照して、 B を決定

2.3 Multi Axes Attention

局所的な特徴と大域的な特徴のそれぞれに焦点を当てた、2種類のAttention

1. Block Attention
2. Grid Attention

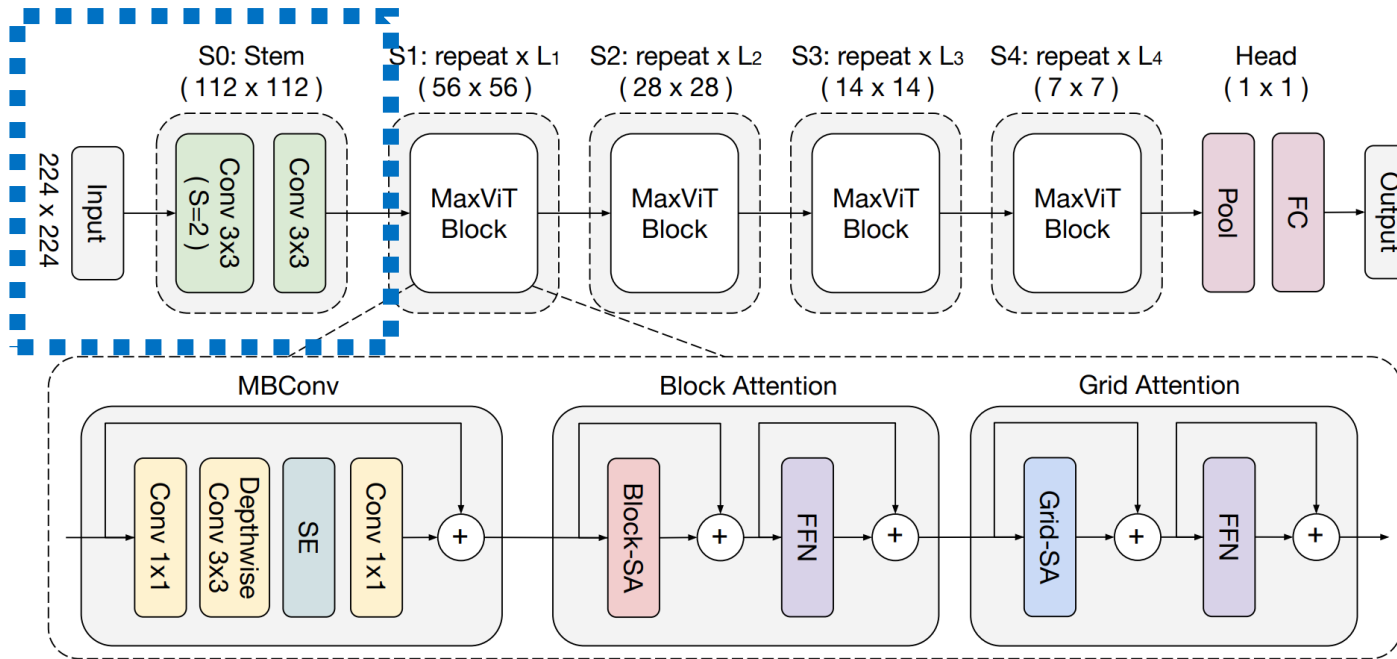


$$\begin{aligned} \text{Block} : (H, W, C) &\rightarrow \left(\frac{H}{P} \times P, \frac{W}{P} \times P, C\right) \\ &\rightarrow \left(\frac{HW}{P^2}, P^2, C\right). \end{aligned}$$

$$\begin{aligned} \text{Grid} : (H, W, C) &\rightarrow \left(G \times \frac{H}{G}, G \times \frac{W}{G}, C\right) \\ &\rightarrow \underbrace{\left(G^2, \frac{HW}{G^2}, C\right)}_{\text{swapaxes(axis1=-2,axis2=-3)} \rightarrow \left(\frac{HW}{G^2}, G^2, C\right) \quad (5) \end{aligned}$$

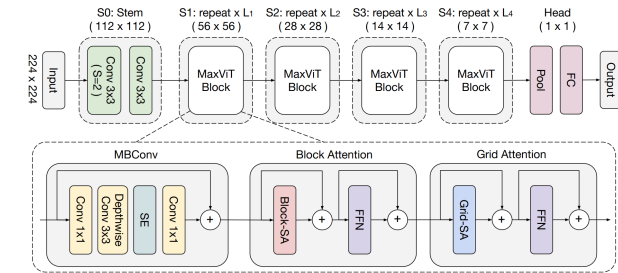
2.4 パッチ化の処理

バニラのViTは、画像をパッチ化していたが、畳み込みによる特徴量に変化



2.5 MBConv

各MaxViTブロックの中で、MBConvブロックを使用

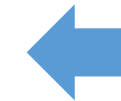
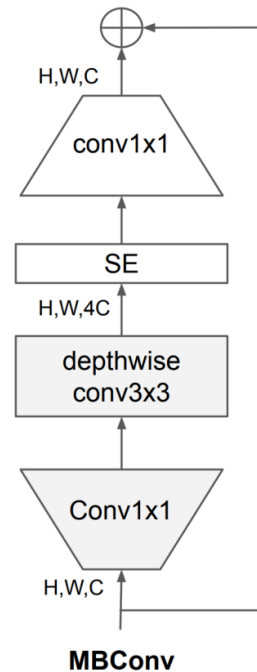


EfficientNet・EfficientNetV2

反転残差ブロックの内部を改良したMBConvを使用

- 1×1畳み込み層(pointwise convolution)でチャンネル数を大きくする
- depthwise convolutionを使用することで計算量を抑える
- SEを用いることで重要な情報を強調する
- もう一度1×1畳み込み層によってチャンネル数を元に戻す
- 各層の後にBN、Swishを適用している。ただし加算する直前ではSwishは適用しない(ReLU関数と同様の理由)

※ $\text{Swish}(x) = x \cdot (1 + \exp(-\beta x))^{-1}$
 β はハイパーパラメータ。形がReLUに似ている



先週のEfficeintNetでの
宮本さんのスライド